



**Course 1504302 Image Processing and Application**

**License Plate Recognition Using SVM**

**Presented by**

- |                                             |                   |
|---------------------------------------------|-------------------|
| <b>1. Mr. Nattakit Phaisanwatchalakoson</b> | <b>6631502036</b> |
| <b>2. Mr. Natchanon Ariyakulthorn</b>       | <b>6631502037</b> |
| <b>3. Mr. Phubadin Wongchai</b>             | <b>6631502043</b> |

**Present to**

**Asst.Prof. Sirikan Chucherd, Ph.D.**

**Semester 1 Academic year 2025**

**Mae Fah Luang university**

## **Preface**

This project, developed for the course 1504302 Image Processing and Application, is titled “Smart Image Processing Toolbox for Real-World Applications.” It aims to explore the practical use of image processing techniques in solving real-world problems.

Our specific focus is on License Plate Recognition using Support Vector Machine (SVM), an application widely used in intelligent transportation systems. The project demonstrates how digital image processing, feature extraction, and machine learning can be integrated to automatically detect and recognize vehicle license plates with accuracy and efficiency.

We hope that this project will contribute to future research and serve as a useful reference for those interested in combining image processing, machine learning, and graphical interface design for real-world applications.

## **Lists of content**

|                                                                 |              |
|-----------------------------------------------------------------|--------------|
| <b>1. Preface</b>                                               | <b>1</b>     |
| <b>2. Lists of content</b>                                      | <b>2</b>     |
| <b>3. Module 1: Image Enhancement</b>                           | <b>3-4</b>   |
| <b>4. Module 2: Image Segmentation and Edge Detection</b>       | <b>5-9</b>   |
| <b>5. Module 3: Geometric Transformations and Interpolation</b> | <b>10-12</b> |
| <b>6. Result</b>                                                | <b>13</b>    |
| <b>7. Conclusion</b>                                            | <b>14</b>    |

## Module 1: Image Enhancement

### Objective

The purpose of the Image Enhancement stage is to improve image quality and emphasize relevant features to support reliable license plate detection. This process reduces noise, increases contrast, and enhances edges to make the plate area more distinguishable from the background.

### Methodology

#### Step 1: Image Loading and Resizing

```
img = cv2.imread("car3.jpg")  
img = cv2.resize(img, (800, int(img.shape[0] * 800 / img.shape[1])))
```

Maintains aspect ratio while resizing to a width of 800 pixels.

#### Step 2: Grayscale Conversion

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

Reduces three color channels to one intensity channel, focusing on brightness rather than color information.

#### Step 3: Noise Reduction (Bilateral Filtering)

```
gray = cv2.bilateralFilter(gray, 11, 17, 17)
```

Removes image noise while preserving sharp edges.

#### Step 4: Edge Detection (Canny Algorithm)

```
edges = cv2.Canny(gray, 50, 50)
```

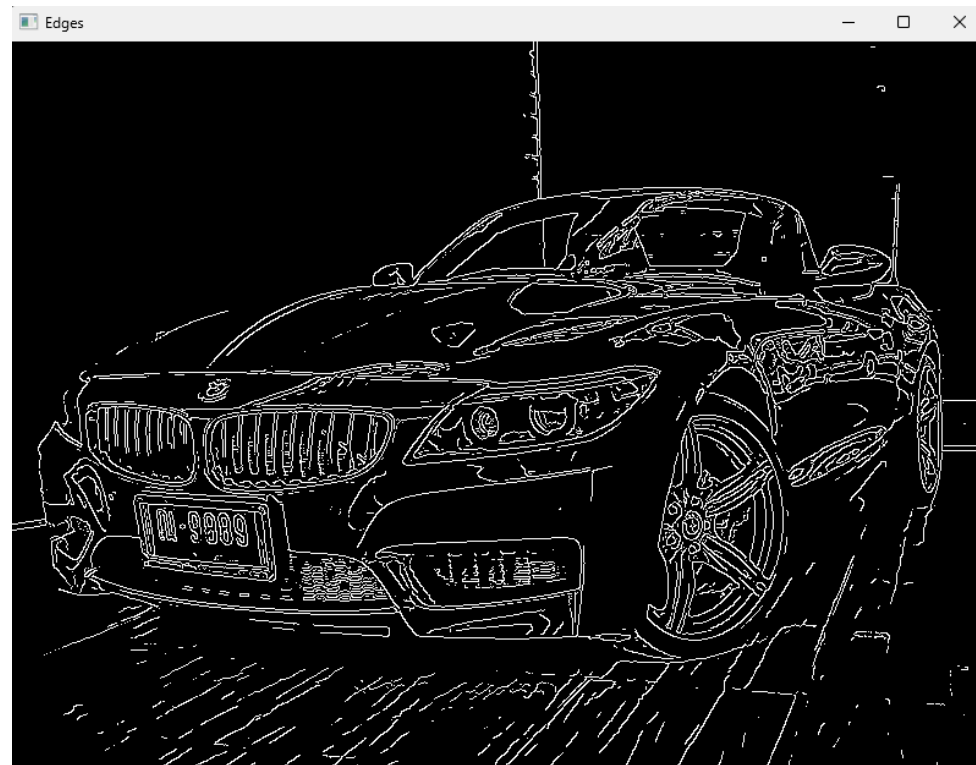
For highlights plate outlines.

## Output

```
cv2.imshow("Edges", edges)  
cv2.imshow("Original", img)
```

Edges Image: Displays detected boundaries.

Original Image: Provides a reference for comparison.



## **Module 2: Image Segmentation and Edge Detection**

### **Introduction**

This module focuses on detecting edges, segmenting regions, and refining images to isolate license plates. These steps are crucial for accurate recognition using SVM. Python, OpenCV, and Matplotlib were used to implement and visualize the process.

### **1. Edge Detection**

Edge detection highlights boundaries in the image. Three methods are used:

Sobel: Computes horizontal and vertical gradients, then combines them to get edge intensity.

Prewitt: Uses custom kernels for gradient calculation and combines them for edges.

Canny: Detects edges using gradient, non-maximum suppression, and thresholding.

The `compare_edges` function visualizes all three methods for comparison.

### **2. Image Segmentation**

Segmentation isolates regions of interest, such as the license plate.

Otsu Thresholding: Automatically computes a threshold to binarize the image.

Region Growing: Expands a region from a seed pixel based on intensity similarity.

Color Segmentation: Masks pixels within a color range in RGB or HSV space.

### **3. Morphological Operations**

Morphological operations refine segmented images:

Dilation: Expands white regions.

Erosion: Shrinks white regions.

Opening: Removes small noise.

Closing: Fills small holes.

These help clean the license plate region before recognition.

## 4. Main Demonstration

Loads the input image.

Applies edge detection, segmentation, and morphological operations.

Visualizes all results in a grid for easy comparison.

## 5. Codes

```

5  # =====
6  # EDGE DETECTION
7  # =====
8
9  def sobel_edge(img):
10     """Edge detection using Sobel operator."""
11     gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
12     grad_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
13     grad_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
14     sobel = cv2.magnitude(grad_x, grad_y)
15     return np.uint8(np.clip(sobel, 0, 255))
16
17 def prewitt_edge(img):
18     """Edge detection using Prewitt operator."""
19     gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
20     kernelx = np.array([[1, 0, -1], [1, 0, -1], [1, 0, -1]])
21     kernely = np.array([[1, 1, 1], [0, 0, 0], [-1, -1, -1]])
22     grad_x = cv2.filter2D(gray, -1, kernelx)
23     grad_y = cv2.filter2D(gray, -1, kernely)
24     prewitt = cv2.magnitude(np.float32(grad_x), np.float32(grad_y))
25     return np.uint8(np.clip(prewitt, 0, 255))
26
27 def canny_edge(img, low=100, high=200):
28     """Edge detection using Canny operator."""
29     gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
30     return cv2.Canny(gray, low, high)
31
32 def compare_edges(img):
33     """Compare Sobel, Prewitt, and Canny edge detection results."""
34     sobel = sobel_edge(img)
35     prewitt = prewitt_edge(img)
36     canny = canny_edge(img)
37
38     titles = ['Original', 'Sobel', 'Prewitt', 'Canny']
39     images = [img, sobel, prewitt, canny]
40
41     plt.figure(figsize=(12, 6))
42     for i in range(4):
43         plt.subplot(1, 4, i + 1)
44         if len(images[i].shape) == 2:
45             plt.imshow(images[i], cmap='gray')
46         else:
47             plt.imshow(cv2.cvtColor(images[i], cv2.COLOR_BGR2RGB))
48         plt.title(titles[i])
49         plt.axis('off')
50     plt.tight_layout()
51     plt.show()

```

```

54 # =====
55 # IMAGE SEGMENTATION
56 # =====
57 def otsu_threshold(img):
58     """Segment using Otsu's thresholding."""
59     gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
60     blur = cv2.GaussianBlur(gray, (5, 5), 0)
61     _, thresh = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
62     return thresh
63
64 def region_growing(img, seed=None, threshold=15):
65     """Simple region growing segmentation."""
66     gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
67     if seed is None:
68         seed = (gray.shape[0] // 2, gray.shape[1] // 2)
69     region = np.zeros_like(gray)
70     stack = [seed]
71     region_mean = gray[seed]
72     region_pixels = 1
73
74     while stack:
75         x, y = stack.pop()
76         region[x, y] = 255
77         for dx in [-1, 0, 1]:
78             for dy in [-1, 0, 1]:
79                 xn, yn = x + dx, y + dy
80                 if (0 <= xn < gray.shape[0]) and (0 <= yn < gray.shape[1]) and region[xn, yn] == 0:
81                     if abs(int(gray[xn, yn]) - int(region_mean)) < threshold:
82                         region[xn, yn] = 255
83                         stack.append((xn, yn))
84                         region_mean = (region_mean * region_pixels + gray[xn, yn]) / (region_pixels + 1)
85                         region_pixels += 1
86     return region
87
88 def color_segmentation(img, color_space='HSV'):
89     """Segment image by color in RGB or HSV space."""
90     if color_space == 'HSV':
91         converted = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
92         lower = np.array([0, 0, 100]) # Example: light color range
93         upper = np.array([179, 80, 255])
94     else: # RGB
95         converted = img
96         lower = np.array([100, 100, 100])
97         upper = np.array([255, 255, 255])
98
99     mask = cv2.inRange(converted, lower, upper)
100     segmented = cv2.bitwise_and(img, img, mask=mask)
101     return segmented

```



```

103 # =====
104 # MORPHOLOGICAL OPERATIONS
105 # =====
106
107 def morphological_ops(img):
108     """Apply dilation, erosion, opening, and closing."""
109     kernel = np.ones((5, 5), np.uint8)
110     dilation = cv2.dilate(img, kernel, iterations=1)
111     erosion = cv2.erode(img, kernel, iterations=1)
112     opening = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel)
113     closing = cv2.morphologyEx(img, cv2.MORPH_CLOSE, kernel)
114     return dilation, erosion, opening, closing
115

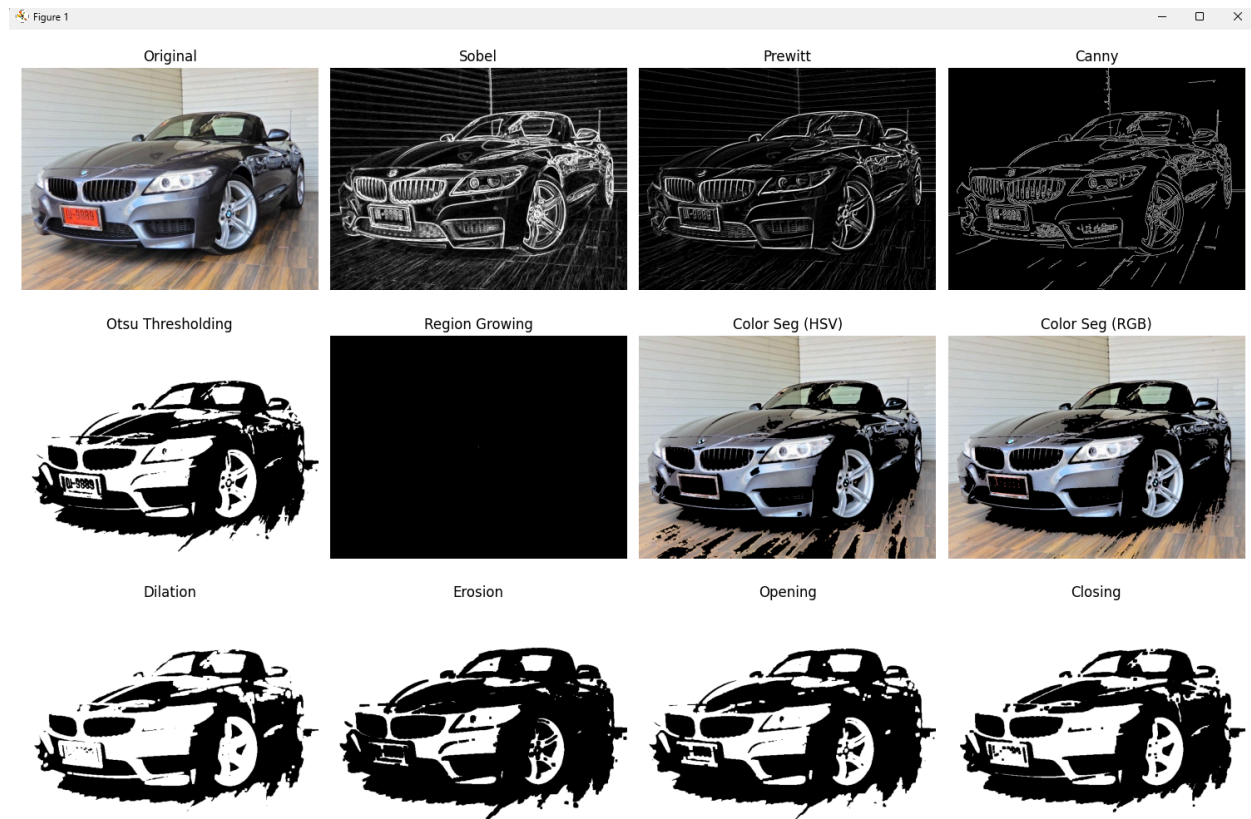
```

```

117 # =====
118 # MAIN DEMONSTRATION
119 # =====
120
121 if __name__ == "__main__":
122     # Load image
123     img = cv2.imread("car3.jpg")
124
125     # ---- Edge Detection ----
126     sobel = sobel_edge(img)
127     prewitt = prewitt_edge(img)
128     canny = canny_edge(img)
129
130     # ---- Segmentation ----
131     otsu = otsu_threshold(img)
132     region = region_growing(img)
133     color_hsv = color_segmentation(img, 'HSV')
134     color_rgb = color_segmentation(img, 'RGB')
135
136     # ---- Morphological Processing ----
137     dilation, erosion, opening, closing = morphological_ops(otsu)
138
139     # ---- Visualization ----
140     titles = [
141         'Original', 'Sobel', 'Prewitt', 'Canny',
142         'Otsu Thresholding', 'Region Growing', 'Color Seg (HSV)', 'Color Seg (RGB)',
143         'Dilation', 'Erosion', 'Opening', 'Closing'
144     ]
145     images = [
146         img, sobel, prewitt, canny,
147         otsu, region, color_hsv, color_rgb,
148         dilation, erosion, opening, closing
149     ]
150
151     plt.figure(figsize=(15, 10))
152     for i in range(len(images)):
153         plt.subplot(3, 4, i + 1)
154         if len(images[i].shape) == 2:
155             plt.imshow(images[i], cmap='gray')
156         else:
157             plt.imshow(cv2.cvtColor(images[i], cv2.COLOR_BGR2RGB))
158         plt.title(titles[i])
159         plt.axis('off')
160     plt.tight_layout()
161     plt.show()
162

```

## 6. Result



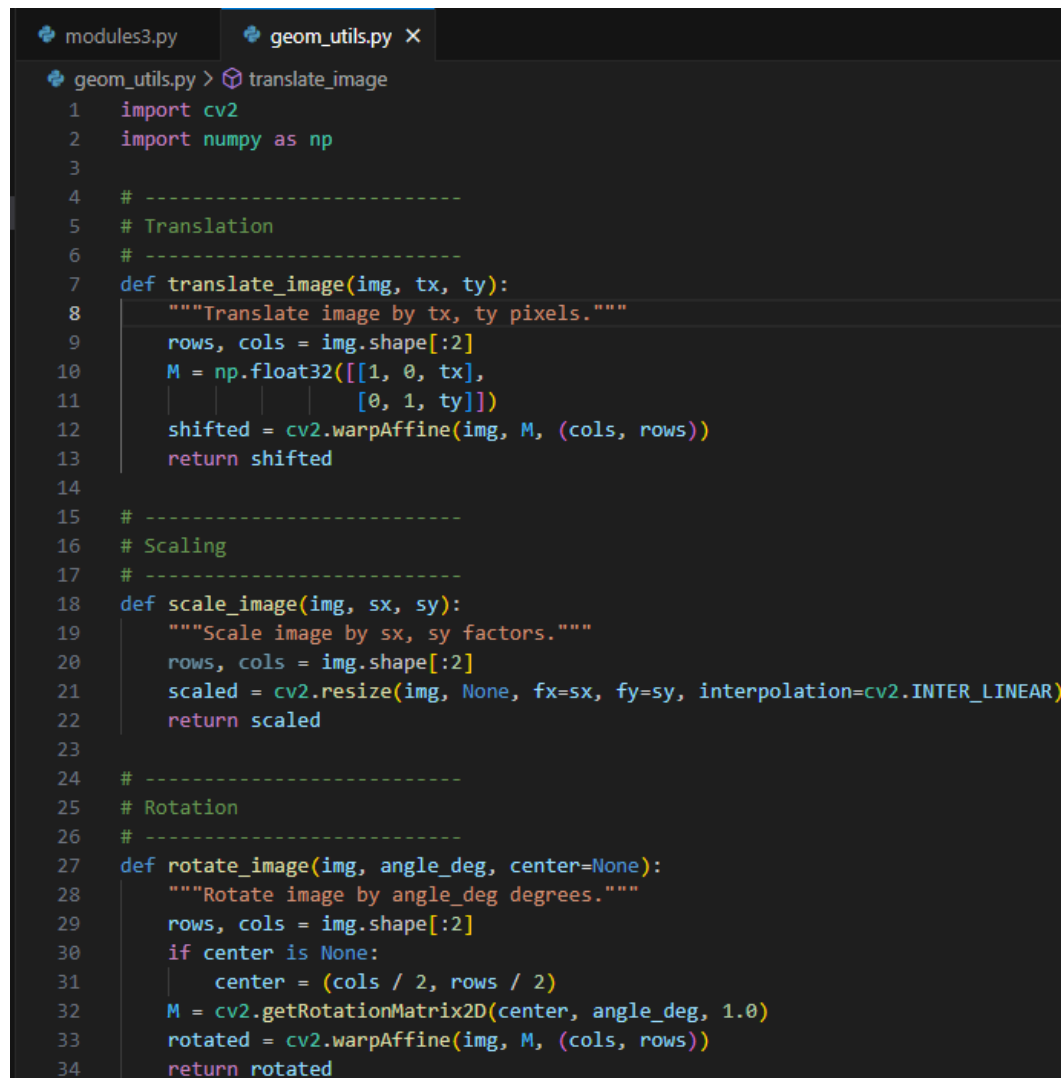
## Module 3: Geometric Transformations and Interpolation

The goal of this experiment is to perform and analyze geometric transformations and interpolation techniques. These operations are essential in computer vision tasks for image alignment, normalization, and data augmentation, particularly in systems such as License Plate Recognition.

**The implementation was divided into two Python files:**

### (1)geom\_utils.py

This file defines basic functions for geometric transformations and interpolation using OpenCV and NumPy.



```

modules3.py  geom_utils.py X
geom_utils.py > translate_image
1  import cv2
2  import numpy as np
3
4  # -----
5  # Translation
6  # -----
7  def translate_image(img, tx, ty):
8      """Translate image by tx, ty pixels."""
9      rows, cols = img.shape[:2]
10     M = np.float32([[1, 0, tx],
11                    [0, 1, ty]])
12     shifted = cv2.warpAffine(img, M, (cols, rows))
13     return shifted
14
15 # -----
16 # Scaling
17 # -----
18 def scale_image(img, sx, sy):
19     """Scale image by sx, sy factors."""
20     rows, cols = img.shape[:2]
21     scaled = cv2.resize(img, None, fx=sx, fy=sy, interpolation=cv2.INTER_LINEAR)
22     return scaled
23
24 # -----
25 # Rotation
26 # -----
27 def rotate_image(img, angle_deg, center=None):
28     """Rotate image by angle_deg degrees."""
29     rows, cols = img.shape[:2]
30     if center is None:
31         center = (cols / 2, rows / 2)
32     M = cv2.getRotationMatrix2D(center, angle_deg, 1.0)
33     rotated = cv2.warpAffine(img, M, (cols, rows))
34     return rotated

```

## (2)modules3.py

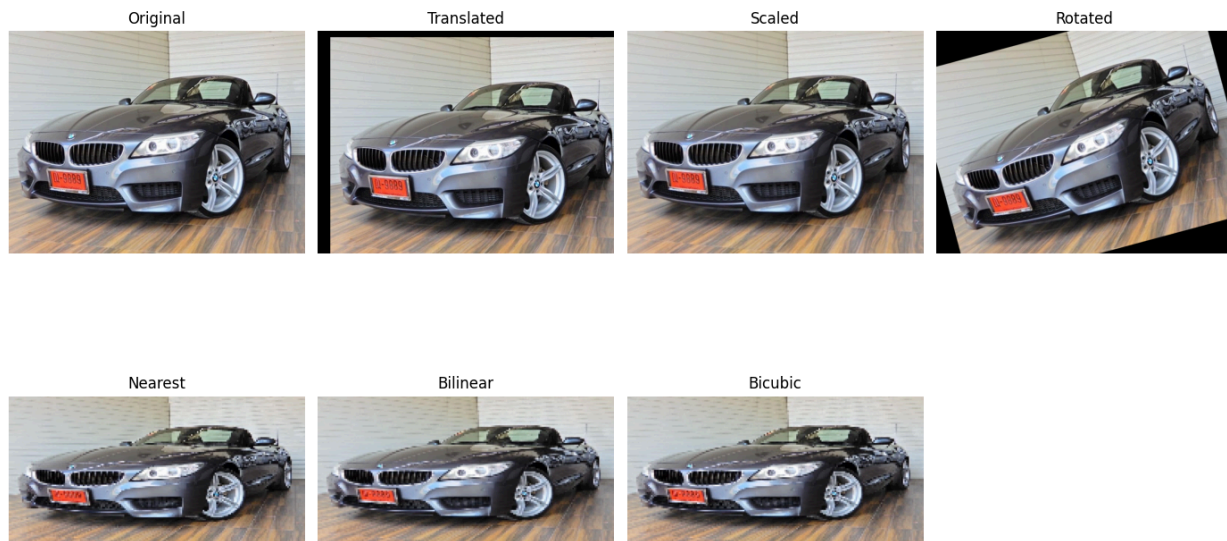
Executes all transformations and displays results.

**Sample main script:**

```
modules3.py • geom_utils.py
modules3.py > ...
1  import cv2
2  import matplotlib.pyplot as plt
3  from geom_utils import (
4      translate_image, scale_image, rotate_image,
5      resize_nearest, resize_bilinear, resize_bicubic_cv
6  )
7
8  img = cv2.imread("car3.jpg")
9  img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
10
11  translated = translate_image(img, tx=30, ty=15)
12  scaled = scale_image(img, sx=1.5, sy=1.5)
13  rotated = rotate_image(img, angle_deg=15)
14
15  small_nearest = resize_nearest(img, (100, 200))
16  small_bilinear = resize_bilinear(img, (100, 200))
17  small_bicubic = resize_bicubic_cv(img, (100, 200))
18
19  titles = ["Original", "Translated", "Scaled", "Rotated", "Nearest", "Bilinear", "Bicubic"]
20  images = [img, translated, scaled, rotated, small_nearest, small_bilinear, small_bicubic]
21
22  plt.figure(figsize=(14, 8))
23  for i, (title, im) in enumerate(zip(titles, images)):
24      plt.subplot(2, 4, i + 1)
25      plt.imshow(im)
26      plt.title(title)
27      plt.axis("off")
28  plt.tight_layout()
29  plt.show()
```

## Result Analysis:

The figure that displays the outputs of various geometric transformations and interpolation methods applied to the sample image of a car. The top row demonstrates translation, scaling, and rotation, while the bottom row compares nearest, bilinear, and bicubic interpolation results.



**Geometric transformations** were used to adjust the position, size, and angle of the image.

The translation moved the image right and down, the scaling enlarged it by 1.5 times, and the rotation tilted it 15 degrees.

Each transformation worked correctly and showed how images can be repositioned or resized without losing quality.

**Interpolation Techniques** affect the visual smoothness and accuracy of resized images.

Nearest-neighbor → fastest, but lowest fidelity.

Bilinear → balanced speed and smoothness.

Bicubic → best overall visual quality, preserving clarity in the car's texture and license plate.

## License Plate Recognition Using SVM



The program has a simple user interface. You can choose a car image from your computer. The system will find and zoom in on the license plate. It will then show the letters and numbers it reads. Finally, it also displays the accuracy of the recognition.

## Conclusion

This project has shown how image processing techniques can be combined with Support Vector Machine (SVM) to recognize vehicle license plates effectively. By breaking the process into modules—image enhancement, segmentation with edge detection, and geometric transformations—we created a structured workflow that improved image clarity, isolated the plate region, and prepared the data for recognition. Each step contributed to making the license plate more distinguishable and ready for accurate classification.

The enhancement stage reduced noise and emphasized important features, while segmentation and edge detection helped locate the plate area precisely. Geometric transformations and interpolation then ensured that the plate images were aligned and consistent, which is critical for reliable recognition. Together, these preprocessing steps provided a strong foundation for the SVM classifier to perform well.

The SVM model successfully recognized the characters on the license plate and reported accuracy levels, demonstrating the importance of combining machine learning with careful image preparation. The results highlight how preprocessing directly impacts recognition performance, showing that even simple methods can achieve strong outcomes when applied systematically.

Overall, this project illustrates the potential of integrating image processing and machine learning for intelligent transportation systems. While the current system achieves good accuracy, future improvements could involve deep learning models, larger datasets, and real-time processing to enhance performance further. This work serves as a practical example of how academic research can be applied to real-world problems and can inspire future developments in smart image processing applications.